

The background features a dark, textured surface with several bright, glowing lines. A central vertical line is yellow, while the lines on either side are red. These lines curve downwards and outwards, creating a stylized, symmetrical shape that resembles a person's legs or a wide, open structure. The overall effect is futuristic and high-tech.

Computational Thinking

A. BHULAI

Procedure

- ❑ We will discuss the problem
- ❑ You work together in pairs
- ❑ 5 -10 minutes per problem
- ❑ We discuss the solution

Problem: Neurons

If there is a system of neurons in which each neuron is directly connected to every other neuron (once), and the total number of connections is 861, how many neurons are there in the system?

Solution: Neurons

Nodes: neurons

Edges: connections (861)

Complete graph: $\frac{n \times (n-1)}{2}$

$$\frac{n \times (n-1)}{2} = 861$$

$$\Rightarrow n \times (n-1) = 1722$$

$$\Rightarrow n^2 - n - 1722 = 0$$

Using the formula for solving quadratic equations $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$:

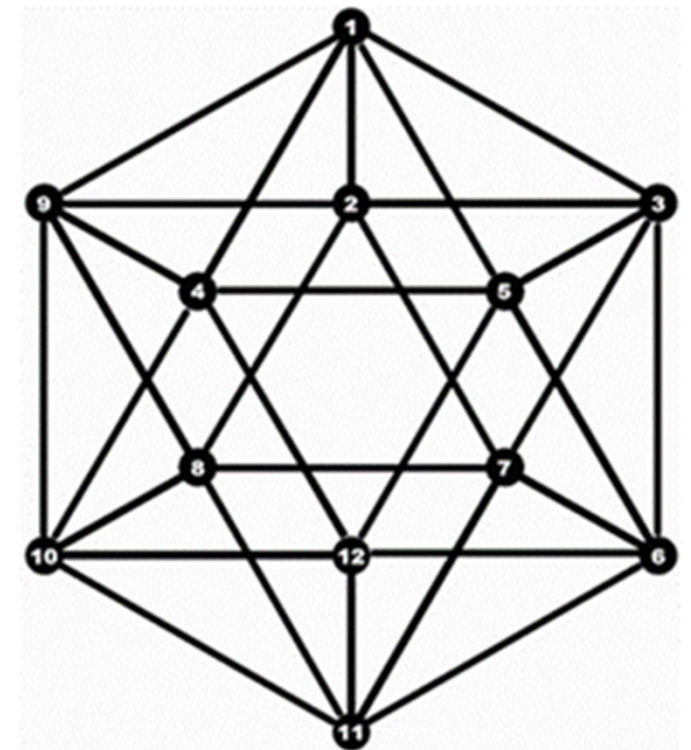
$$n_1 = 42$$

$$n_2 = -41$$

Problem: Model to simulate the spread of viruses

A group of scientists are developing a model to simulate the spread of viruses. The models are graphs which show the social activities of people during a given time period. One such graph can be seen here. Each vertex denotes a person, and a connecting edge means that two people have met each other during the given period – which is when the virus may pass from one person to another.

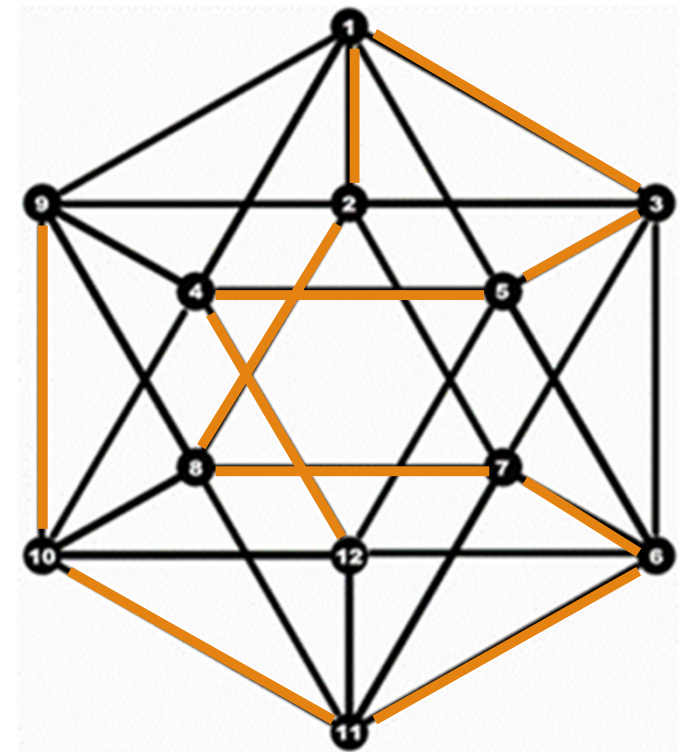
The scientists are experimenting with different patterns of virus spread. Given the graph model on the right, show if it is possible for a virus to reach every person in the model if only one person has it in the beginning and each person passes it on to only one other.



Solution: Model to simulate the spread of viruses

Use the Hamiltonian path:

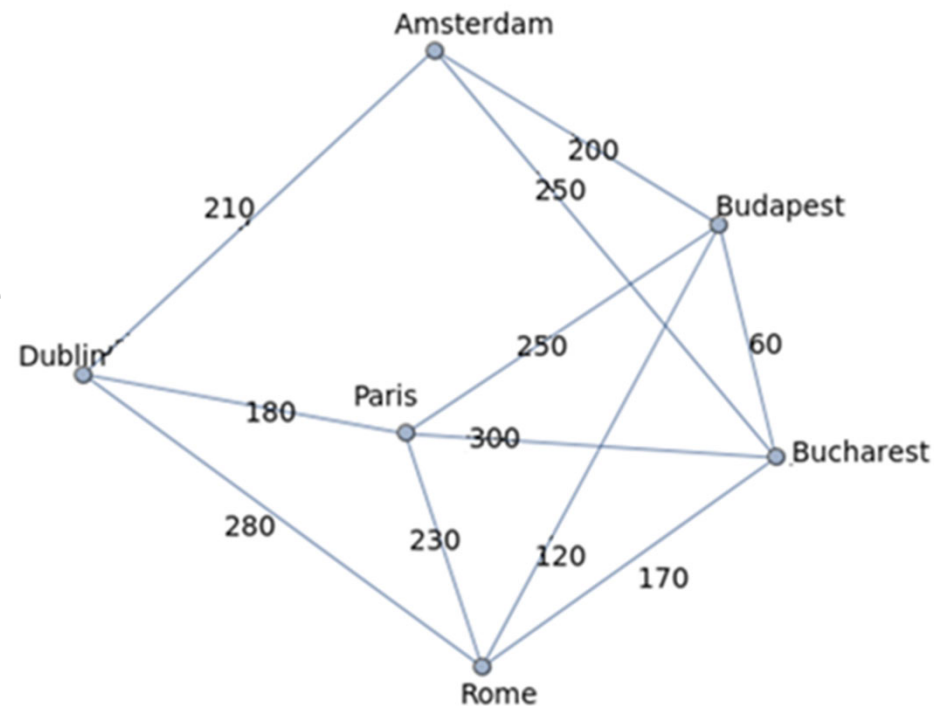
9 – 10 – 11 – 6 – 7 – 8 – 2 – 1 – 3 – 5 – 4 – 12



Problem: Schedule flights

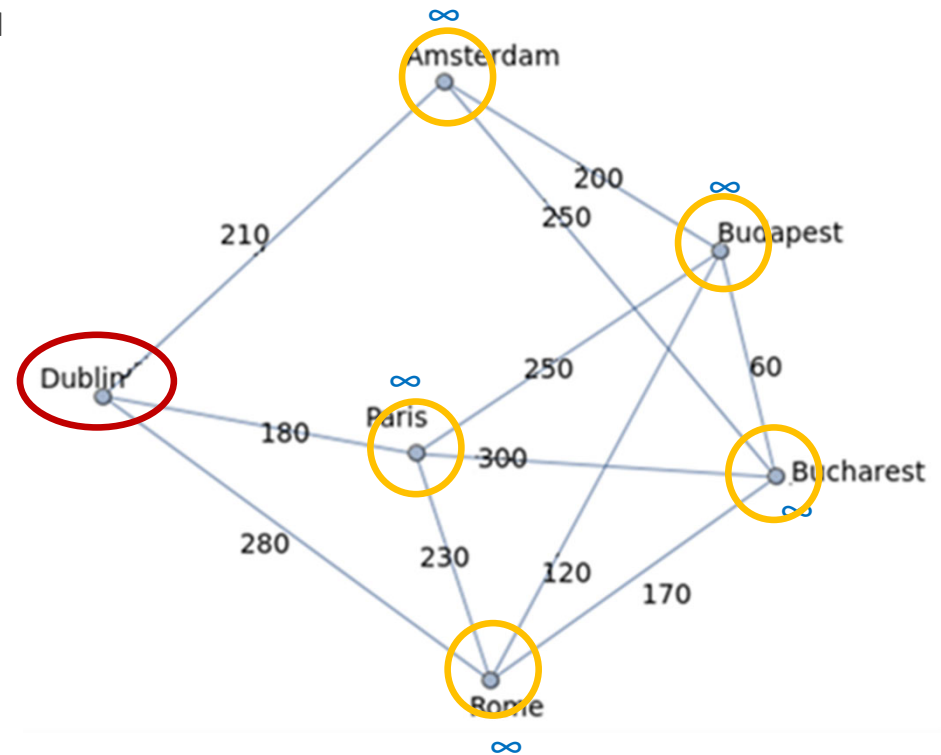
Flight scheduling websites allow users to filter itineraries based on different criteria. One of the most popular filters is the price of the journey. Imagine that a user needs to travel from Dublin to Bucharest last minute, so that they cannot find a direct flight. The graph here shows the available flights with the prices given in Euros (you can assume that a flight costs the same in both directions between two cities).

Which flights will the system recommend to this user first if they filter for price of the journey between Dublin and Bucharest? Show the intermediate steps of an algorithm the system could use to find the best flights.



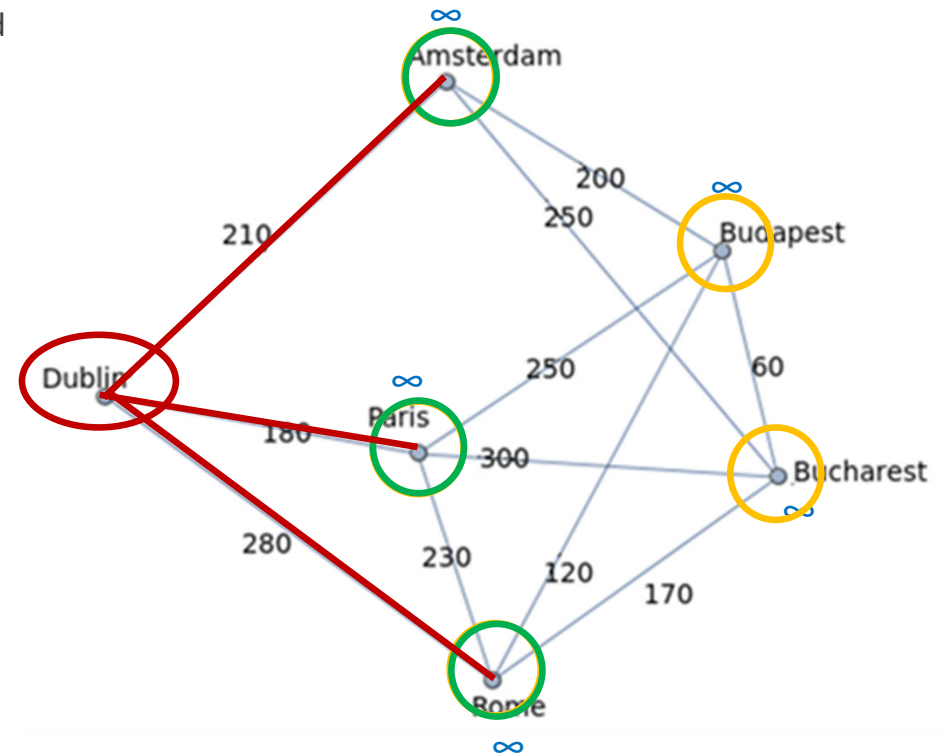
Solution: Schedule flights

1. Initial node: Dublin
2. Current node: Dublin. All other nodes are marked unvisited, and an infinity price is assigned to them.



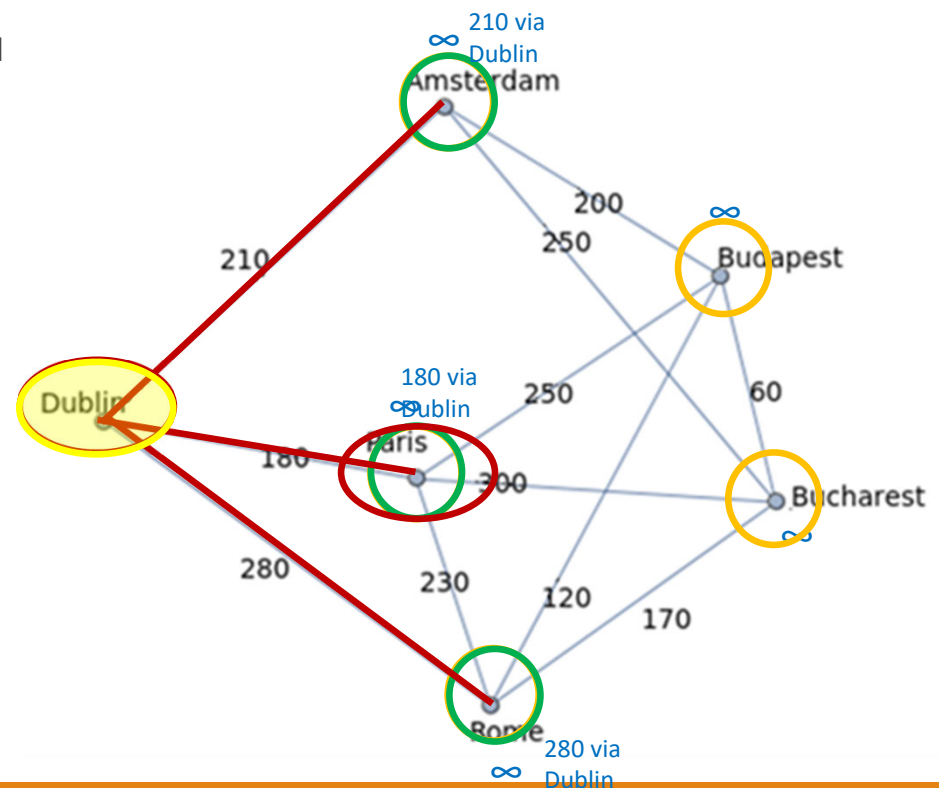
Solution: Schedule flights

1. Initial node: Dublin
2. Current node: Dublin. All other nodes are marked unvisited, and an infinite price is assigned to them.



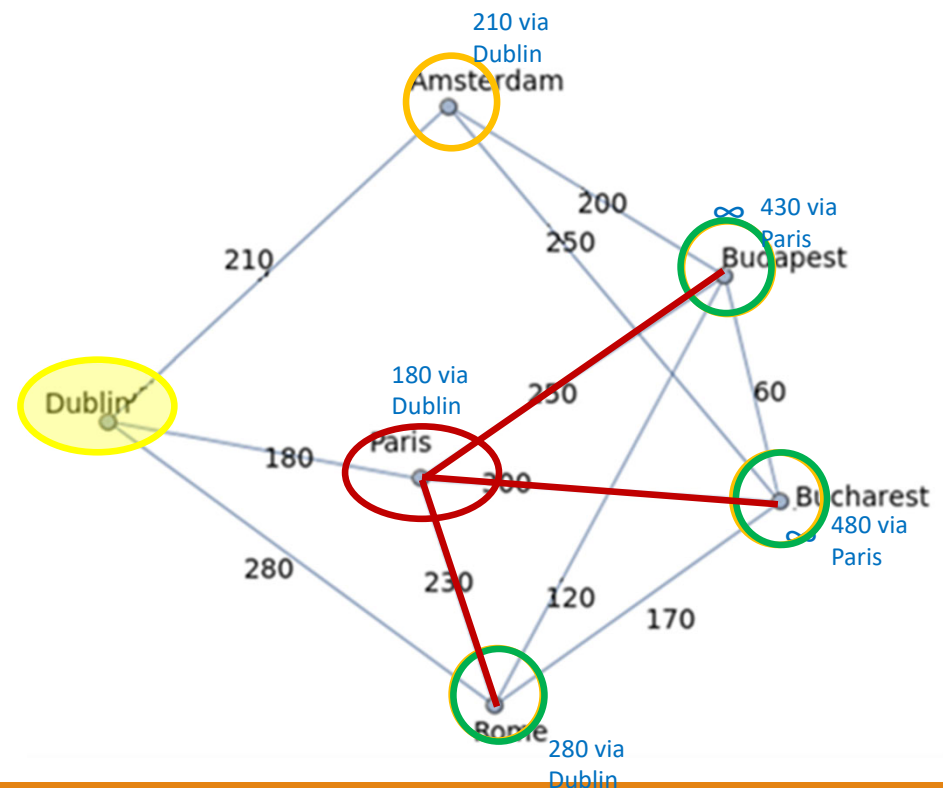
Solution: Schedule flights

1. Initial node: Dublin
2. Current node: Dublin. All other nodes are marked unvisited, and an infinite price is assigned to them.
3. Tentative prices for Dublin's unvisited neighbors:
 - a. Amsterdam 210 (via Dublin)
 - b. Paris 180 (via Dublin)
 - c. Rome 280 (via Dublin)
4. Mark Dublin as visited. Current node: Paris



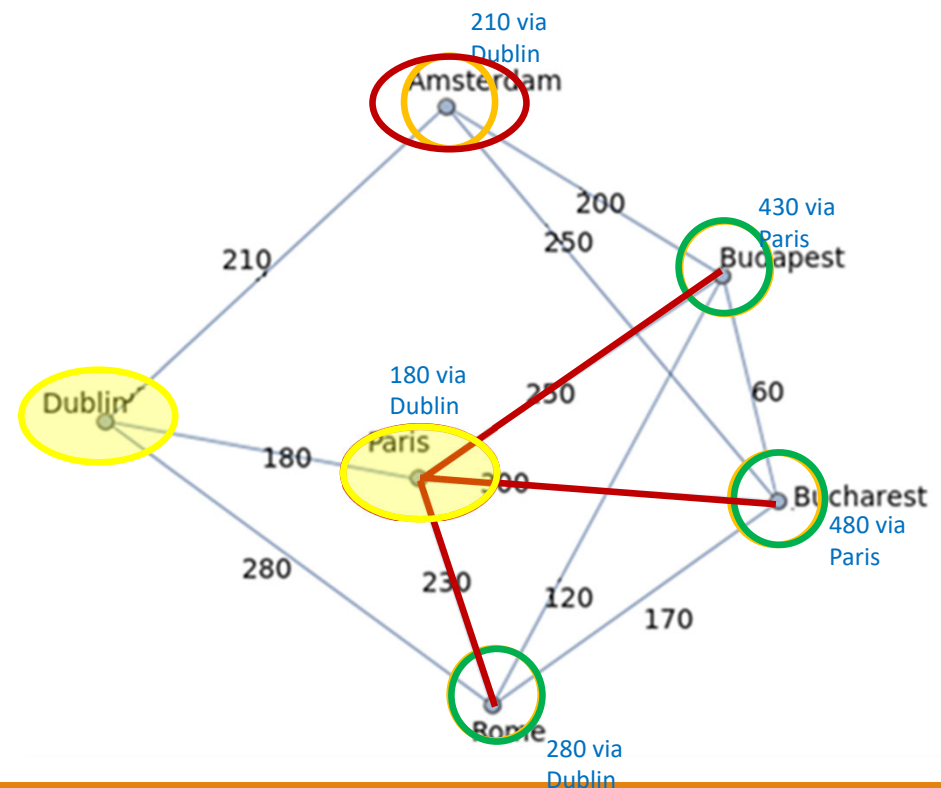
Solution: Schedule flights

1. Initial node: Dublin
2. Current node: Dublin. All other nodes are marked unvisited, and an infinite price is assigned to them.
3. Tentative prices for Dublin's unvisited neighbors:
 - a. Amsterdam 210 (via Dublin)
 - b. Paris 180 (via Dublin)
 - c. Rome 280 (via Dublin)
4. Mark Dublin as visited. Current node: Paris
5. Tentative prices for Paris's unvisited neighbors:
 - a. Budapest 430 (via Paris)
 - b. Bucharest 480 (via Paris)
 - c. Rome 230 (via Paris) – not updated because previous price was cheaper!



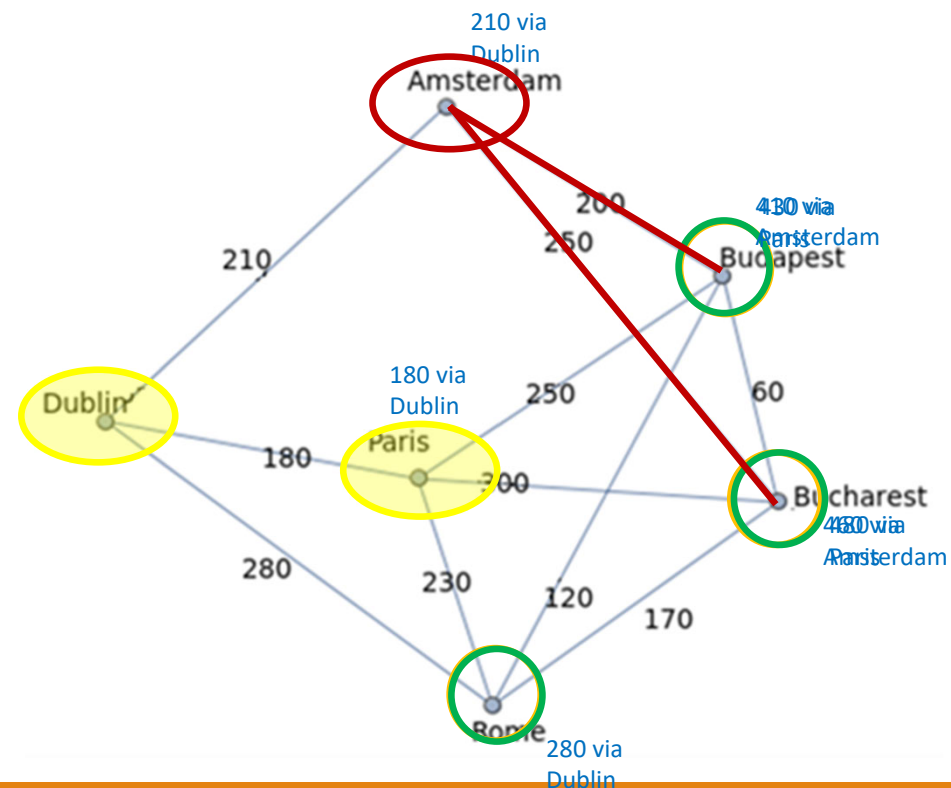
Solution: Schedule flights

1. Initial node: Dublin
2. Current node: Dublin. All other nodes are marked unvisited, and an infinite price is assigned to them.
3. Tentative prices for Dublin's unvisited neighbors:
 - a. Amsterdam 210 (via Dublin)
 - b. Paris 180 (via Dublin)
 - c. Rome 280 (via Dublin)
4. Mark Dublin as visited. Current node: Paris
5. Tentative prices for Paris's unvisited neighbors:
 - a. Budapest 430 (via Paris)
 - b. Bucharest 480 (via Paris)
 - c. Rome 280 (via Dublin) – not updated because previous price was cheaper!
6. Mark Paris as visited. Current node: Amsterdam



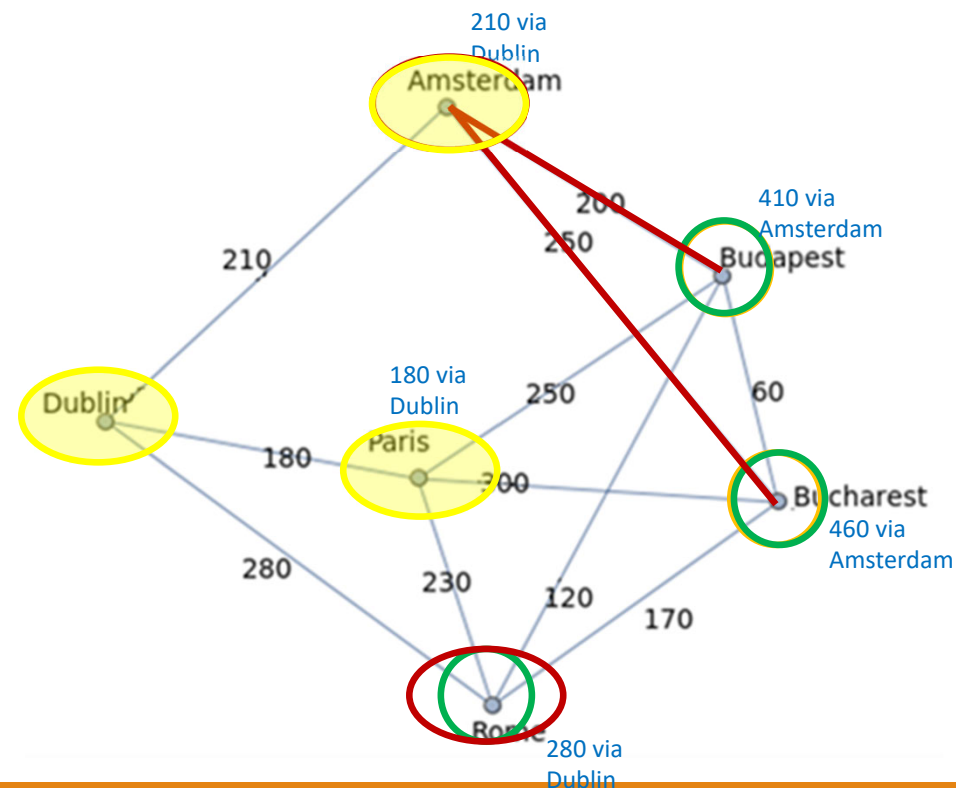
Solution: Schedule flights

1. Initial node: Dublin
2. Current node: Dublin. All other nodes are marked unvisited, and an infinite price is assigned to them.
3. Tentative prices for Dublin's unvisited neighbors:
 - a. Amsterdam 210 (via Dublin)
 - b. Paris 180 (via Dublin)
 - c. Rome 280 (via Dublin)
4. Mark Dublin as visited. Current node: Paris
5. Tentative prices for Paris's unvisited neighbors:
 - a. Budapest 430 (via Paris)
 - b. Bucharest 480 (via Paris)
 - c. Rome 280 (via Dublin) – not updated because previous price was cheaper!
6. Mark Paris as visited. Current node: Amsterdam
7. Tentative prices for Amsterdam's unvisited neighbors:
 - a. Budapest 410 (via Amsterdam)
 - b. Bucharest 460 (via Amsterdam)



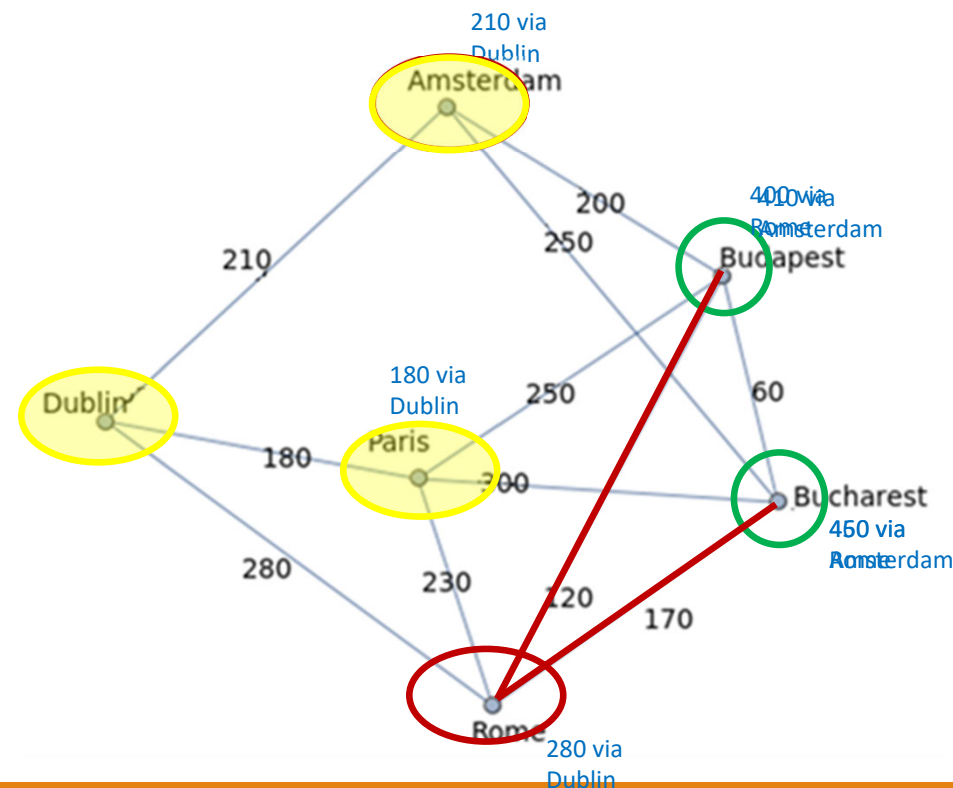
Solution: Schedule flights

1. Initial node: Dublin
2. Current node: Dublin. All other nodes are marked unvisited, and an infinite price is assigned to them.
3. Tentative prices for Dublin's unvisited neighbors:
 - a. Amsterdam 210 (via Dublin)
 - b. Paris 180 (via Dublin)
 - c. Rome 280 (via Dublin)
4. Mark Dublin as visited. Current node: Paris
5. Tentative prices for Paris's unvisited neighbors:
 - a. Budapest 430 (via Paris)
 - b. Bucharest 480 (via Paris)
 - c. Rome 280 (via Dublin) – not updated because previous price was cheaper!
6. Mark Paris as visited. Current node: Amsterdam
7. Tentative prices for Amsterdam's unvisited neighbors:
 - a. Budapest 410 (via Amsterdam)
 - b. Bucharest 460 (via Amsterdam)
8. Mark Amsterdam as visited. Current node: Rome



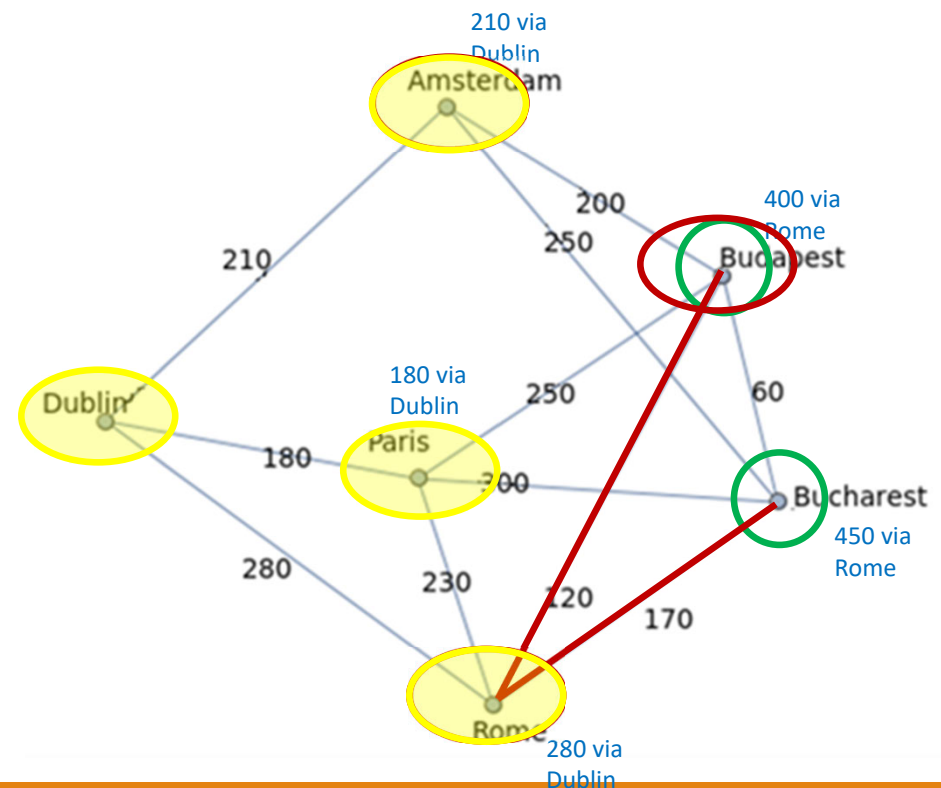
Solution: Schedule flights

1. Initial node: Dublin
2. Current node: Dublin. All other nodes are marked unvisited, and an infinite price is assigned to them.
3. Tentative prices for Dublin's unvisited neighbors:
 - a. Amsterdam 210 (via Dublin)
 - b. Paris 180 (via Dublin)
 - c. Rome 280 (via Dublin)
4. Mark Dublin as visited. Current node: Paris
5. Tentative prices for Paris's unvisited neighbors:
 - a. Budapest 430 (via Paris)
 - b. Bucharest 480 (via Paris)
 - c. Rome 280 (via Dublin) – not updated because previous price was cheaper!
6. Mark Paris as visited. Current node: Amsterdam
7. Tentative prices for Amsterdam's unvisited neighbors:
 - a. Budapest 410 (via Amsterdam)
 - b. Bucharest 460 (via Amsterdam)
8. Mark Amsterdam as visited. Current node: Rome
9. Tentative prices for Rome's unvisited neighbors:
 - a. Budapest 400 (via Rome)
 - b. Bucharest 450 (via Rome)



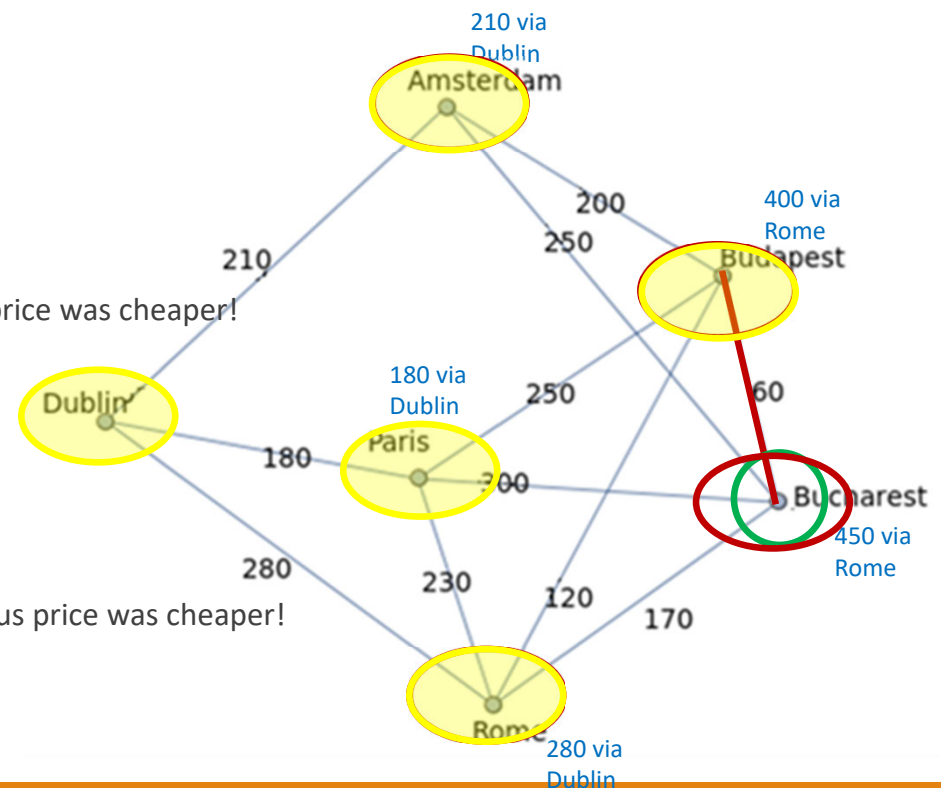
Solution: Schedule flights

1. Initial node: Dublin
2. Current node: Dublin. All other nodes are marked unvisited, and an infinite price is assigned to them.
3. Tentative prices for Dublin's unvisited neighbors:
 - a. Amsterdam 210 (via Dublin)
 - b. Paris 180 (via Dublin)
 - c. Rome 280 (via Dublin)
4. Mark Dublin as visited. Current node: Paris
5. Tentative prices for Paris's unvisited neighbors:
 - a. Budapest 430 (via Paris)
 - b. Bucharest 480 (via Paris)
 - c. Rome 280 (via Dublin) – not updated because previous price was cheaper!
6. Mark Paris as visited. Current node: Amsterdam
7. Tentative prices for Amsterdam's unvisited neighbors:
 - a. Budapest 410 (via Amsterdam)
 - b. Bucharest 460 (via Amsterdam)
8. Mark Amsterdam as visited. Current node: Rome
9. Tentative prices for Rome's unvisited neighbors:
 - a. Budapest 400 (via Rome)
 - b. Bucharest 450 (via Rome)
10. Mark Rome as visited. Current node: Budapest



Solution: Schedule flights

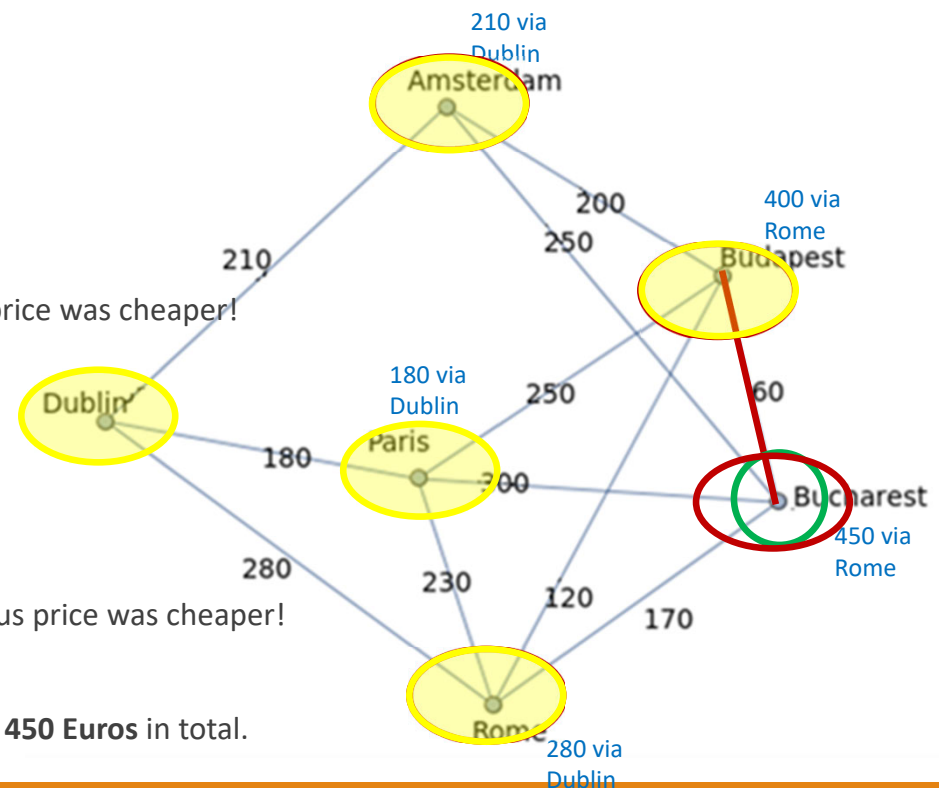
1. Initial node: Dublin
2. Current node: Dublin. All other nodes are marked unvisited, and an infinite price is assigned to them.
3. Tentative prices for Dublin's unvisited neighbors:
 - a. Amsterdam 210 (via Dublin)
 - b. Paris 180 (via Dublin)
 - c. Rome 280 (via Dublin)
4. Mark Dublin as visited. Current node: Paris
5. Tentative prices for Paris's unvisited neighbors:
 - a. Budapest 430 (via Paris)
 - b. Bucharest 480 (via Paris)
 - c. Rome 280 (via Dublin) – not updated because previous price was cheaper!
6. Mark Paris as visited. Current node: Amsterdam
7. Tentative prices for Amsterdam's unvisited neighbors:
 - a. Budapest 410 (via Amsterdam)
 - b. Bucharest 460 (via Amsterdam)
8. Mark Amsterdam as visited. Current node: Rome
9. Tentative prices for Rome's unvisited neighbors:
 - a. Budapest 400 (via Rome)
 - b. Bucharest 450 (via Rome)
10. Mark Rome as visited. Current node: Budapest
11. Tentative prices for Budapest's unvisited neighbors:
 - a. Bucharest 450 (via Rome) – not updated because previous price was cheaper!
12. Mark Budapest as visited. Current node: Bucharest



Solution: Schedule flights

1. Initial node: Dublin
2. Current node: Dublin. All other nodes are marked unvisited, and an infinite price is assigned to them.
3. Tentative prices for Dublin's unvisited neighbors:
 - a. Amsterdam 210 (via Dublin)
 - b. Paris 180 (via Dublin)
 - c. Rome 280 (via Dublin)
4. Mark Dublin as visited. Current node: Paris
5. Tentative prices for Paris's unvisited neighbors:
 - a. Budapest 430 (via Paris)
 - b. Bucharest 480 (via Paris)
 - c. Rome 280 (via Dublin) – not updated because previous price was cheaper!
6. Mark Paris as visited. Current node: Amsterdam
7. Tentative prices for Amsterdam's unvisited neighbors:
 - a. Budapest 410 (via Amsterdam)
 - b. Bucharest 460 (via Amsterdam)
8. Mark Amsterdam as visited. Current node: Rome
9. Tentative prices for Rome's unvisited neighbors:
 - a. Budapest 400 (via Rome)
 - b. Bucharest 450 (via Rome)
10. Mark Rome as visited. Current node: Budapest
11. Tentative prices for Budapest's unvisited neighbors:
 - a. Bucharest 450 (via Rome) – not updated because previous price was cheaper!
12. Mark Budapest as visited. Current node: Bucharest
13. No unvisited neighbors to inspect. Algorithm stops.

Result: the cheapest flight route from Dublin to Bucharest is via Rome, **450 Euros** in total.



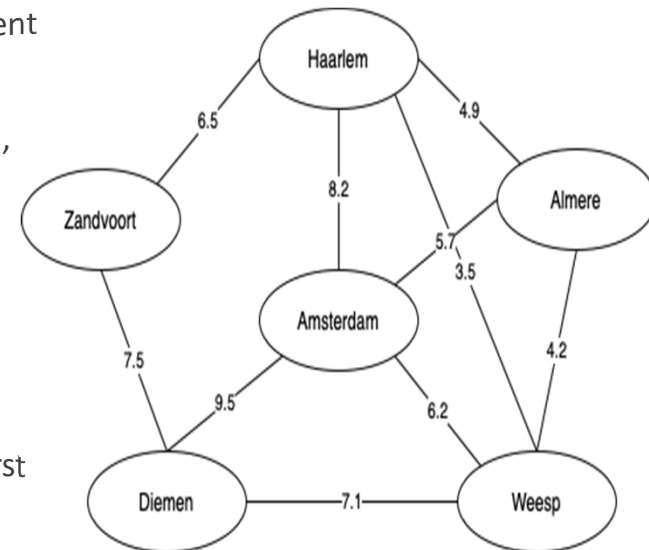
Problem: Renovation project

The Dutch government is planning to launch a large renovation project for bicycle routes connecting different cities in The Netherlands. At the start of the project, they conducted a large-scale survey in which participants could rate the condition of different routes. To decide exactly which routes will be renovated, data from the survey is gathered and digitalized so that a planning software can be used. A sample from the outcome of the survey in the Amsterdam region is represented by the following graph, where the scores on the edges represent the condition score the participants gave to each of the routes between cities (therefore, a lower score means that a route is in poorer condition).

Based on the data, a computer algorithm will decide which routes will be renovated based on two restrictions imposed by the project leaders:

1. Each participating town must be accessible via a renovated route.
2. Only a minimum number of routes must be renovated, and they must be in the worst possible condition (i.e., their overall condition scores must add to the lowest possible amount).

From the sample above, which routes will the system choose for renovation? Name and show the intermediate steps of two algorithms the system could use to decide.

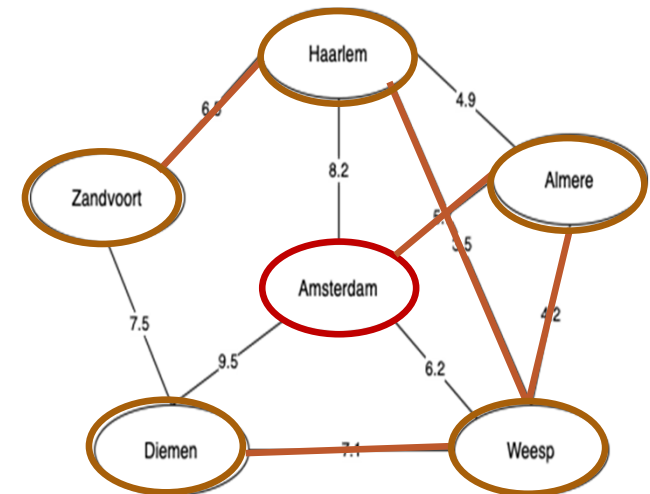


Solution: Renovation project (Prim)

PRIM'S ALGORITHM

1. Initial vertex in T (subgraph of the minimum spanning tree): Amsterdam
2. Edge with the least weight connecting T to other vertices:
Amsterdam – Almere (5.7)
3. Edge with the least weight connecting T to other vertices:
Almere – Weesp (4.2)
4. Edge with the least weight connecting T to other vertices:
Weesp – Haarlem (3.5)
5. Edge with the least weight connecting T to other vertices:
Haarlem – Zandvoort (6.5)
6. Edge with the least weight connecting T to other vertices:
Weesp – Diemen (7.1)
7. T includes every vertex in the graph. Algorithm stops

1. Select any vertex to be the first of T.
2. Consider which edge connects vertices in T to vertices outside T. Pick 1 with minimum weight (if more than 1, choose any). Add this edge and vertex to T.
3. Repeat step 2 until T contains every vertex of the graph.



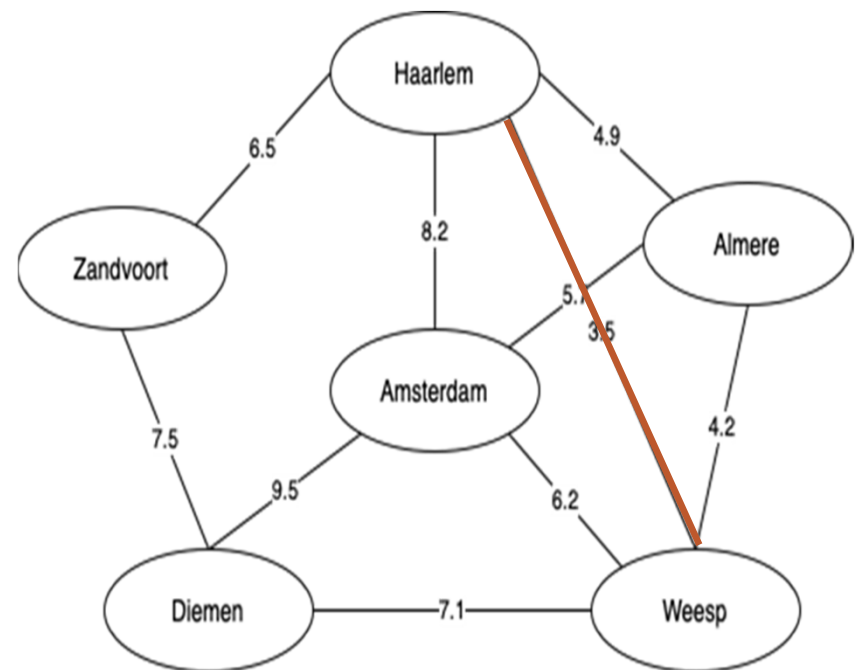
Solution: Renovation project (Kruskal 1/5)

KRUSKAL'S ALGORITHM

1. Remove all loops.
2. Remove all parallel edges between two vertices except the one with least weight.
3. Create an edge table.
4. Select the edge with the minimum weight.
5. Select the next edge with the minimum weight that does not form any cycle with previously selected edges.
6. **Repeat step 5 until T contains every vertex of the graph.**

Weesp -
Haarlem

3.5

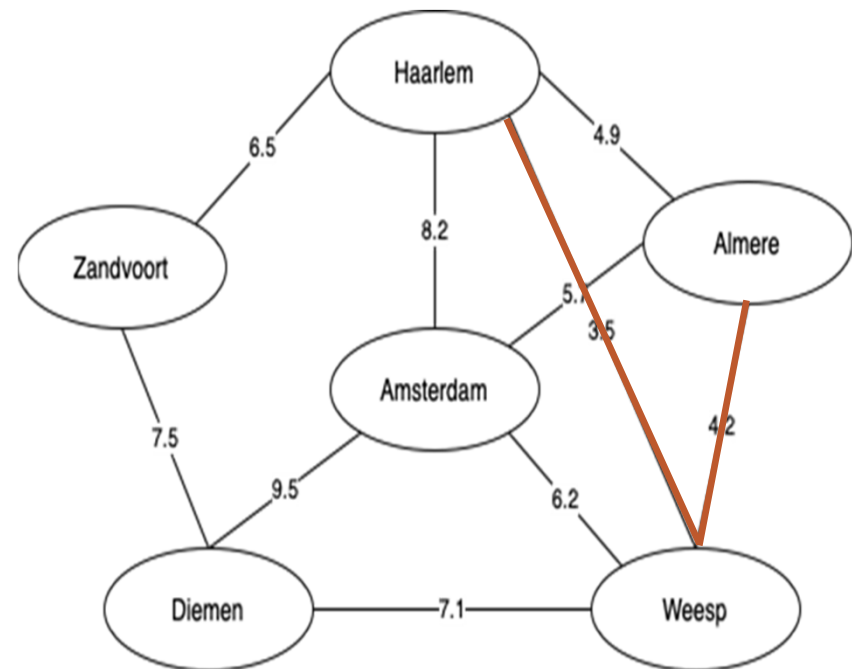


Solution: Renovation project (Kruskal 2/5)

KRUSKAL'S ALGORITHM

1. Remove all loops.
2. Remove all parallel edges between two vertices except the one with least weight.
3. Create an edge table.
4. Select the edge with the minimum weight.
5. Select the next edge with the minimum weight that does not form any cycle with previously selected edges.
6. **Repeat step 5 until T contains every vertex of the graph.**

Weesp - Haarlem	Almere - Weesp
3.5	4.2

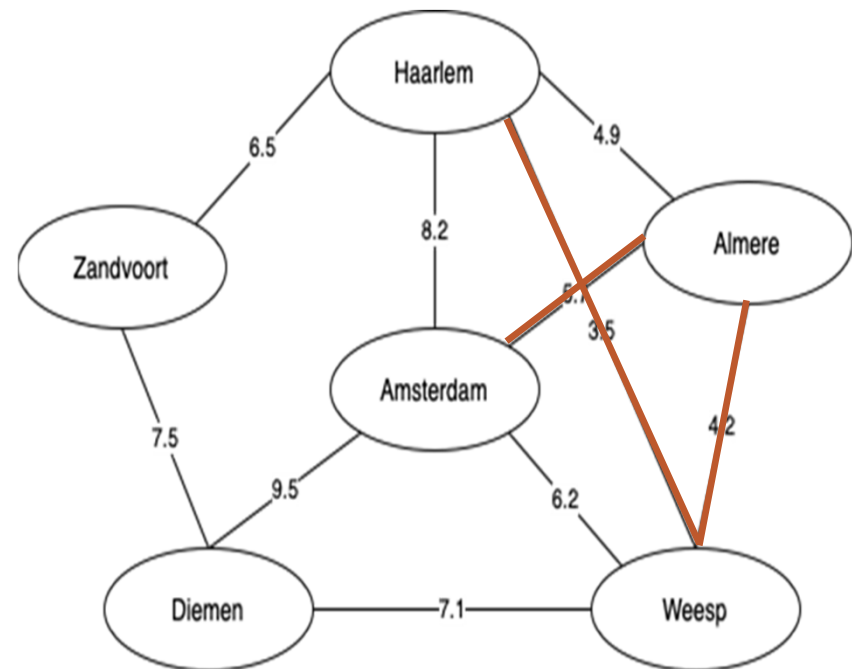


Solution: Renovation project (Kruskal 3/5)

KRUSKAL'S ALGORITHM

1. Remove all loops.
2. Remove all parallel edges between two vertices except the one with least weight.
3. Create an edge table.
4. Select the edge with the minimum weight.
5. Select the next edge with the minimum weight that does not form any cycle with previously selected edges.
6. **Repeat step 5 until T contains every vertex of the graph.**

Weesp - Haarlem	Almere - Weesp	Amsterdam - Almere
3.5	4.2	5.7

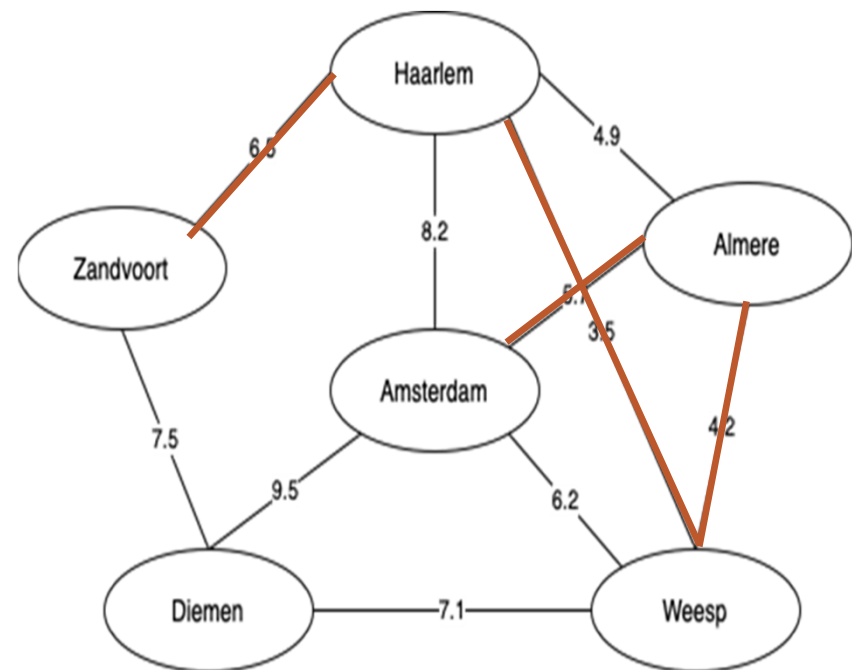


Solution: Renovation project (Kruskal 4/5)

KRUSKAL'S ALGORITHM

1. Remove all loops.
2. Remove all parallel edges between two vertices except the one with least weight.
3. Create an edge table.
4. Select the edge with the minimum weight.
5. Select the next edge with the minimum weight that does not form any cycle with previously selected edges.
6. **Repeat step 5 until T contains every vertex of the graph.**

Weesp - Haarlem	Almere – Weesp	Amsterdam – Almere	Haarlem-Zandvoort
3.5	4.2	5.7	6.5

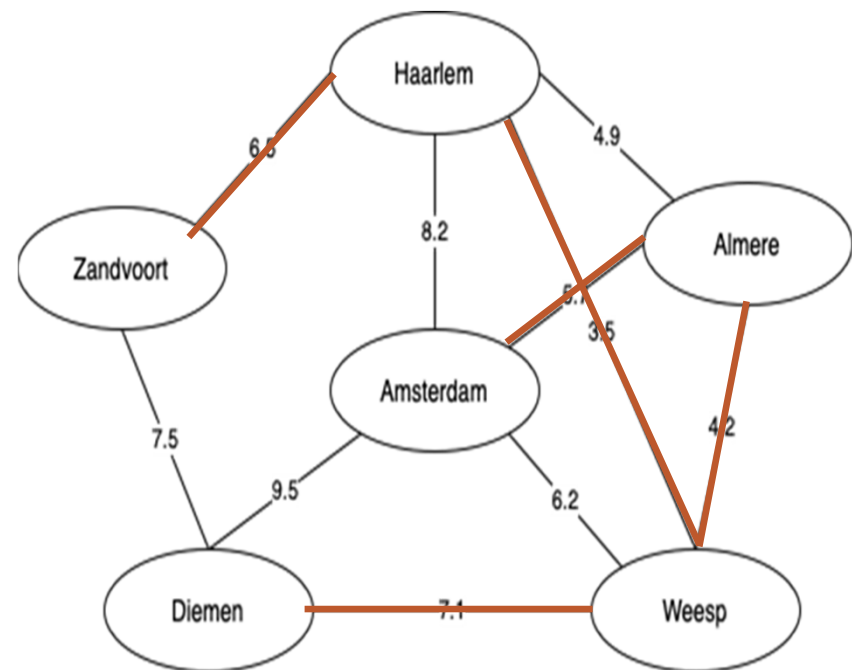


Solution: Renovation project (Kruskal 5/5)

KRUSKAL'S ALGORITHM

1. Remove all loops.
2. Remove all parallel edges between two vertices except the one with least weight.
3. Create an edge table.
4. Select the edge with the minimum weight.
5. Select the next edge with the minimum weight that does not form any cycle with previously selected edges.
6. **Repeat step 5 until T contains every vertex of the graph.**

Weesp - Haarlem	Almere – Weesp	Amsterdam – Almere	Haarlem- Zandvoort	Weesp – Diemen
3.5	4.2	5.7	6.5	7.1



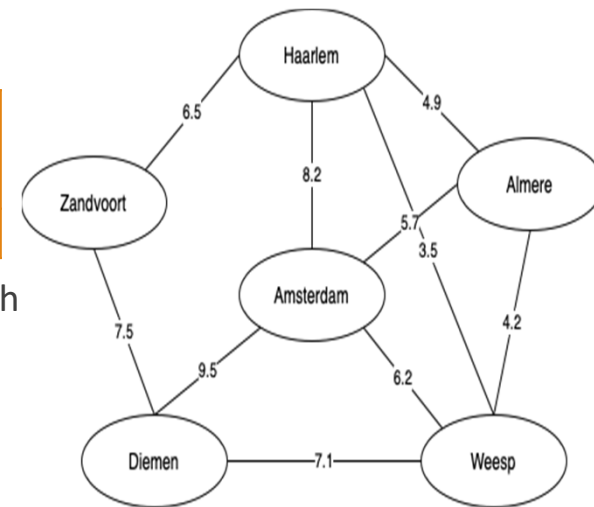
Solution: Renovation project (Kruskal alt.)

KRUSKAL'S ALGORITHM

1. No loops to remove from graph.
2. No double edges to remove from graph.
3. Edge table (can be ordered ascendingly for edge weight as well):

Amsterdam – Almere	Amsterdam – Diemen	Amsterdam – Haarlem	Amsterdam – Weesp	Almere – Weesp	Almere – Haarlem	Haarlem – Zandvoort	Zandvoort – Diemen	Weesp – Diemen	Weesp – Haarlem
5.7	9.5	8.2	6.2	4.2	4.9	6.5	7.5	7.1	3.5

4. Selecting minimum edges from the edge table which do not create a cycle with previously selected edges:
 - * Weesp – Haarlem (3.5)
 - * Almere – Weesp (4.2)
 - * Amsterdam – Almere (5.7)
 - * Haarlem – Zandvoort (6.5)
 - * Weesp – Diemen (7.1)
5. Every vertex in the graph is included in the tree. Algorithm stops.



Solution: Renovation project

PRIM'S ALGORITHM

- Almere – Weesp
- Amsterdam – Almere
- Haarlem – Zandvoort
- Weesp – Diemen
- Weesp - Haarlem

KRUSKAL'S ALGORITHM

- Almere – Weesp
- Amsterdam – Almere
- Haarlem – Zandvoort
- Weesp – Diemen
- Weesp - Haarlem

What have you learned from this course?

- To analyze problems,
- To choose the right solution strategy or a combination of strategies to solve problems,
- To design algorithms,
- To translate algorithms to a flowchart,
- To give an oral presentation of a project to peers,
- To reflect critically on your and other's work,
- To write a project report.

We can't satisfy everyone



IF you liked the course:

evaluate the course positively



ELSE:

give constructive feedback



